EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 11

Refining the Object-Oriented Design

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 11 of 16

Contents

1. Week 11 Introduction

Week 10 introduced Object-Oriented Design and the distinction **analysis = what the system needs, design = how the software is organised**. Week 11 goes one step deeper: **how do we refine those models into a design that developers can actually implement, maintain, test and extend?**

Drawing a UML class diagram is not the same as completing an OO design. This week we refine analysis classes into design classes, refine attributes and operations, assign responsibilities, design relationships, choose association vs inheritance, work with interfaces and abstract classes, manage dependencies, and judge the quality of the resulting design.

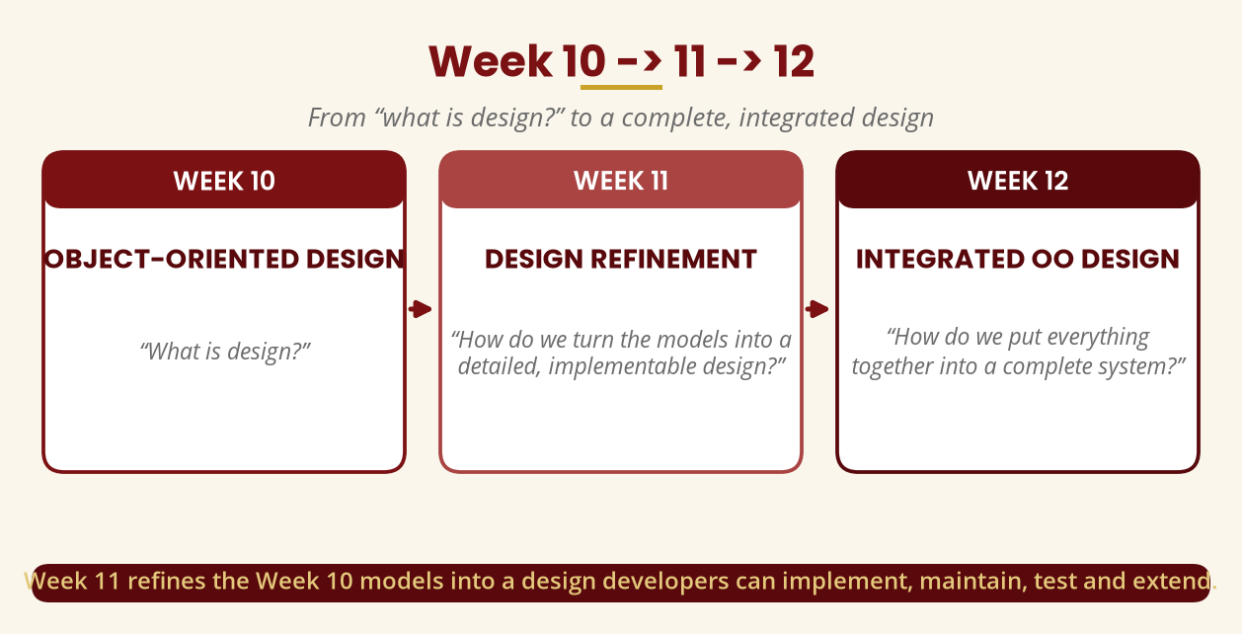

Figure 1 — Week 10 -> 11 -> 12 progression

2. Learning Outcomes

By the end of this week, a student should be able to:

- Explain the meaning of design refinement and the analysis-to-design transformation.
- Distinguish an analysis class from a design class.
- Identify design classes, including entity, boundary and control classes.
- Refine class attributes and operations with types, parameters and return values.
- Assign responsibilities appropriately (responsibility-driven design).
- Choose association vs inheritance using the HAS / IS-A test.
- Explain interfaces and abstract classes, and compare them.
- Explain cohesion, coupling, encapsulation, information hiding, separation of concerns and dependency management.
- Explain layered design, design for reuse, design for change, design quality and traceability.

PART A — WHAT DOES "REFINEMENT" MEAN?

3. The Meaning of Design Refinement

Refinement means taking something general and making it more detailed and precise. Saying *"I want to build a house"* is not enough for a construction team — you eventually need room dimensions, door and window locations, electrical and water layouts, materials, foundation and roof. Software design works the same way: during analysis we identify **Student, Course, Registration**; we then refine them into classes with typed attributes and operations. The design model becomes increasingly precise as it moves toward implementation.

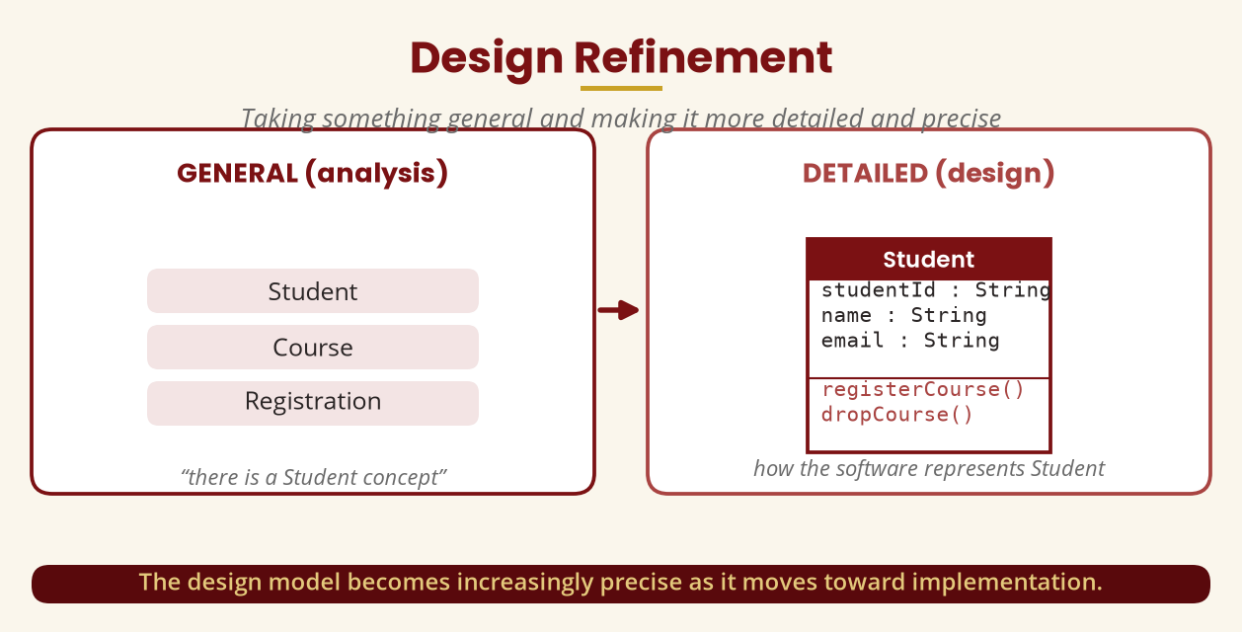

Figure 2 — Refinement: general -> detailed

4. Analysis Class vs Design Class

An **analysis class** represents an important concept in the problem domain (Student, Course, Registration). A **design class** adds the detail required to implement the solution — visibility, attribute types, parameter types, return values and operations. We move from *"there is a Student concept"* to *"here is how the software will represent and manipulate Student."*

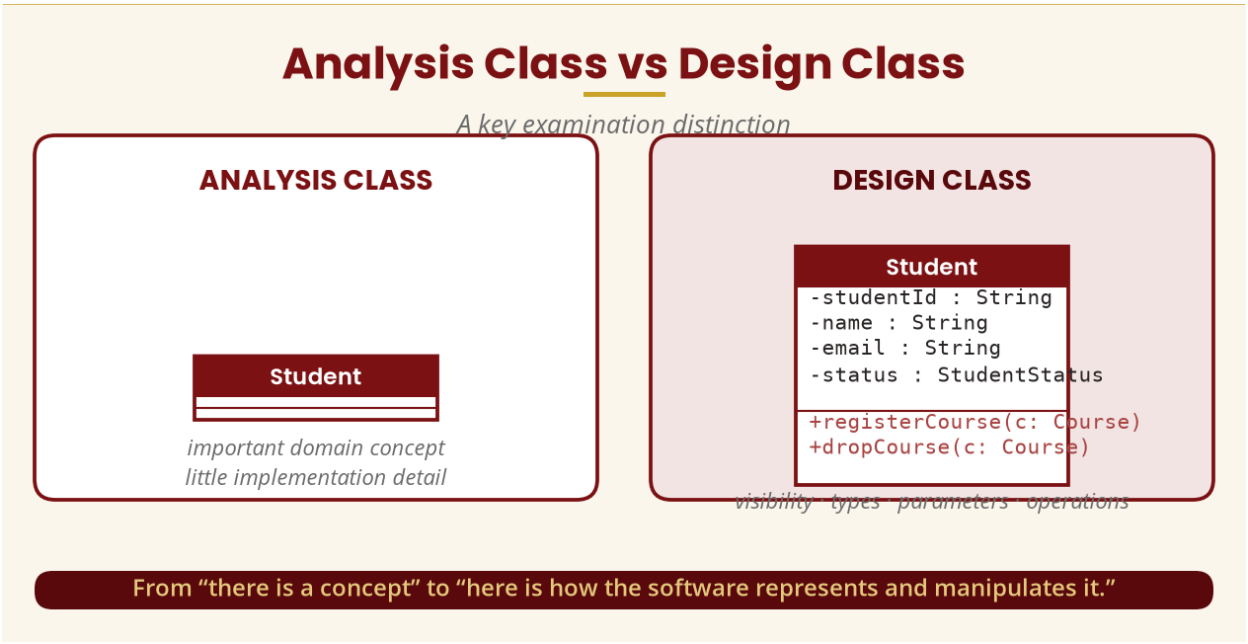

Figure 3 — Analysis class vs design class

PART B — IDENTIFYING DESIGN CLASSES

5. What Is a Design Class?

A **design class** is a class defined with enough structural and behavioural detail to contribute to implementation. It may contain attributes, operations, visibility, types, parameters, return values, relationships, interfaces and constraints. For the requirement *"Students can register for courses online"*, analysis identifies Student, Course and Registration — but design also discovers `RegistrationService`, `StudentRepository`, `CourseRepository`, `RegistrationRepository` and `AuthenticationService`, because design must answer not just *what concepts exist* but *what software components are required to make the system work*.

6. Entity, Boundary and Control Classes

Entity classes represent information important to the business/domain (Student, Course, Registration, Payment, Invoice). **Boundary classes** handle interaction between the system and external actors (LoginPage, RegistrationForm, StudentDashboard, PaymentScreen). **Control classes** coordinate activities or use-case logic (RegistrationController, PaymentController, LoginController) — so we may have Student -> RegistrationPage -> RegistrationController -> Registration -> Course.

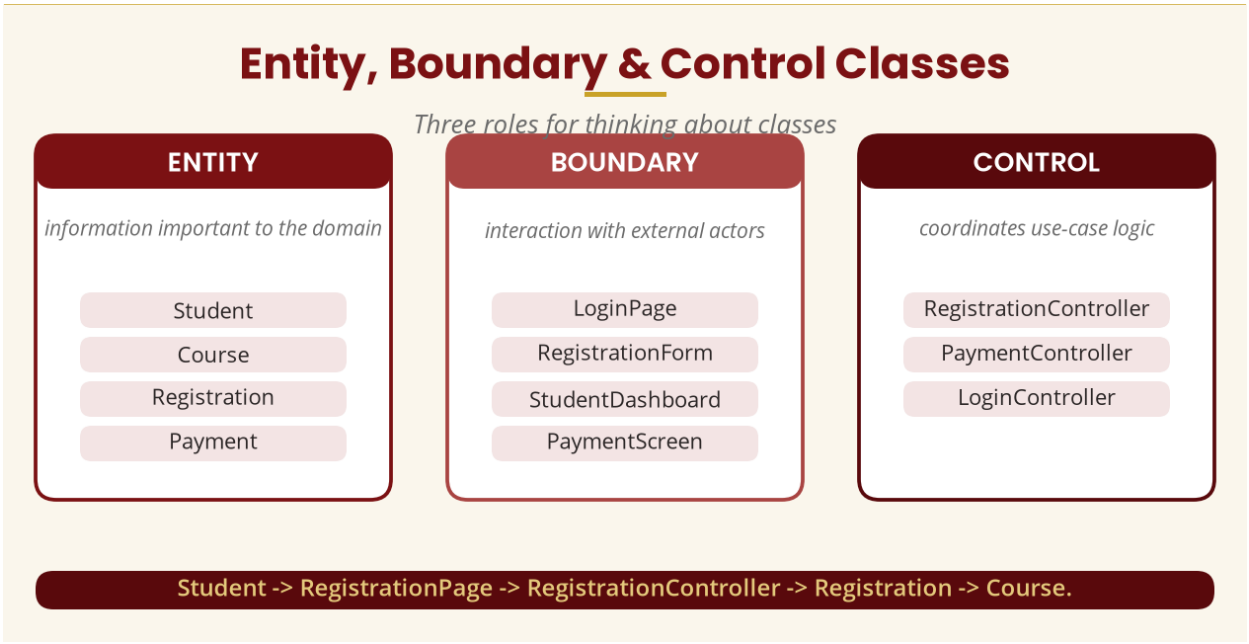

Figure 4 — Entity, boundary and control classes

PART C & D — REFINING ATTRIBUTES AND OPERATIONS

7. Refining Attributes

An **attribute** represents information maintained by an object — `Student` has `studentId`, `name`, `email`, `programme`. Design specifies types: `studentId : String`, `credits : Integer`, `capacity : Integer`. A good designer asks: *"What information does the object actually need to fulfil its responsibilities?"* (not `everything`).

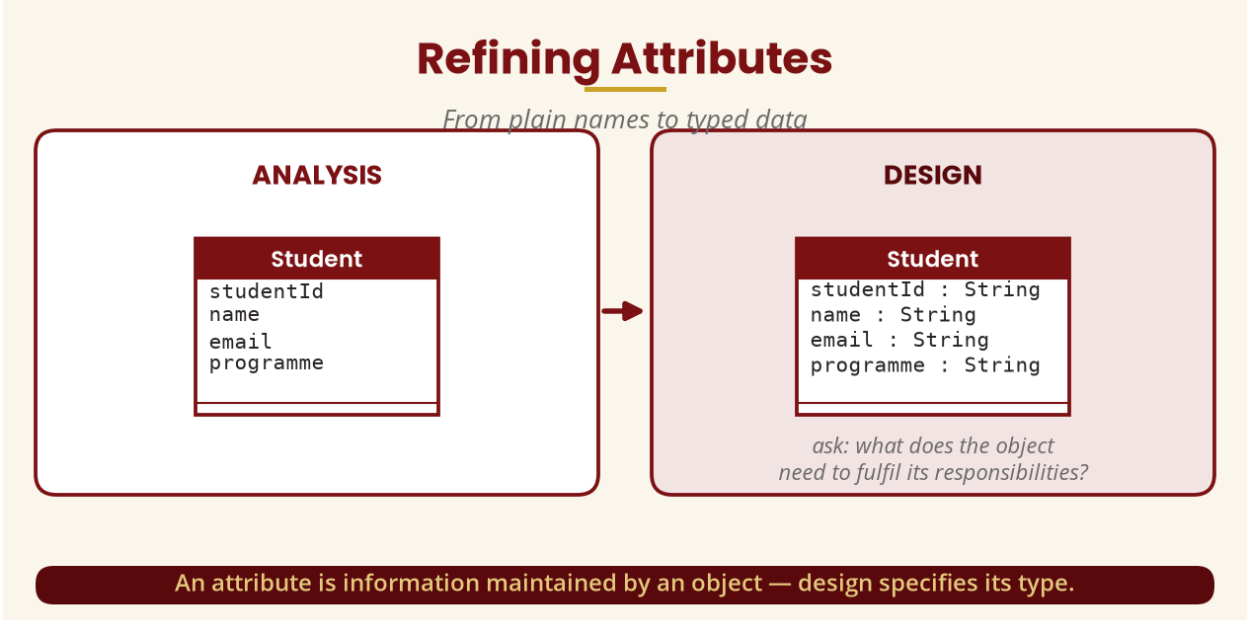

Figure 5 — Refining attributes

8. Refining Operations

An **operation** represents something an object can do. The design model makes it precise: `registerCourse(course : Course) : Registration` tells the developer the operation name, the input (a Course) and the return value (a Registration) — far more useful than a bare `register()`. That is design refinement.

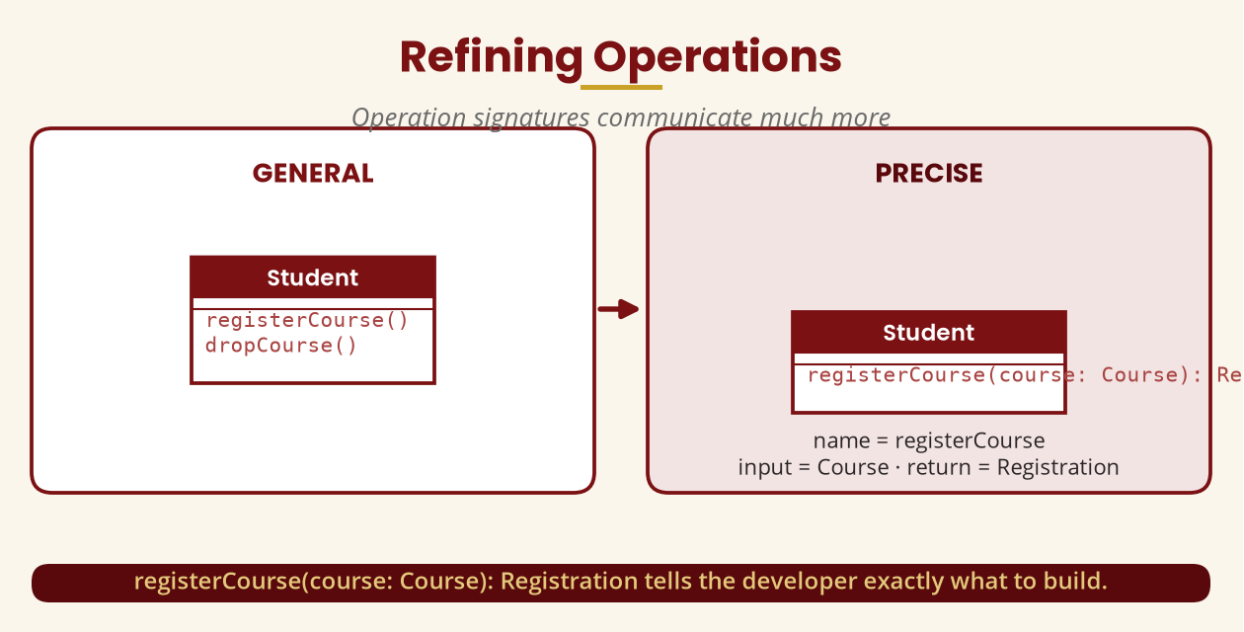

Figure 6 — Refining operations

PART E — RESPONSIBILITY-DRIVEN DESIGN

9. Responsibility Assignment

A major design question is *"Which object should perform a particular operation?"* To check whether a course has space, `Course.hasAvailableSpace()` is more logical than `Student.checkCourseAvailability()`, because availability is a property of the course. As systems grow, registration may involve prerequisite checking, timetable conflicts, financial restrictions, capacity and deadlines — so we introduce a `RegistrationService` to coordinate the process across Student, Course and Registration. That is a design decision.

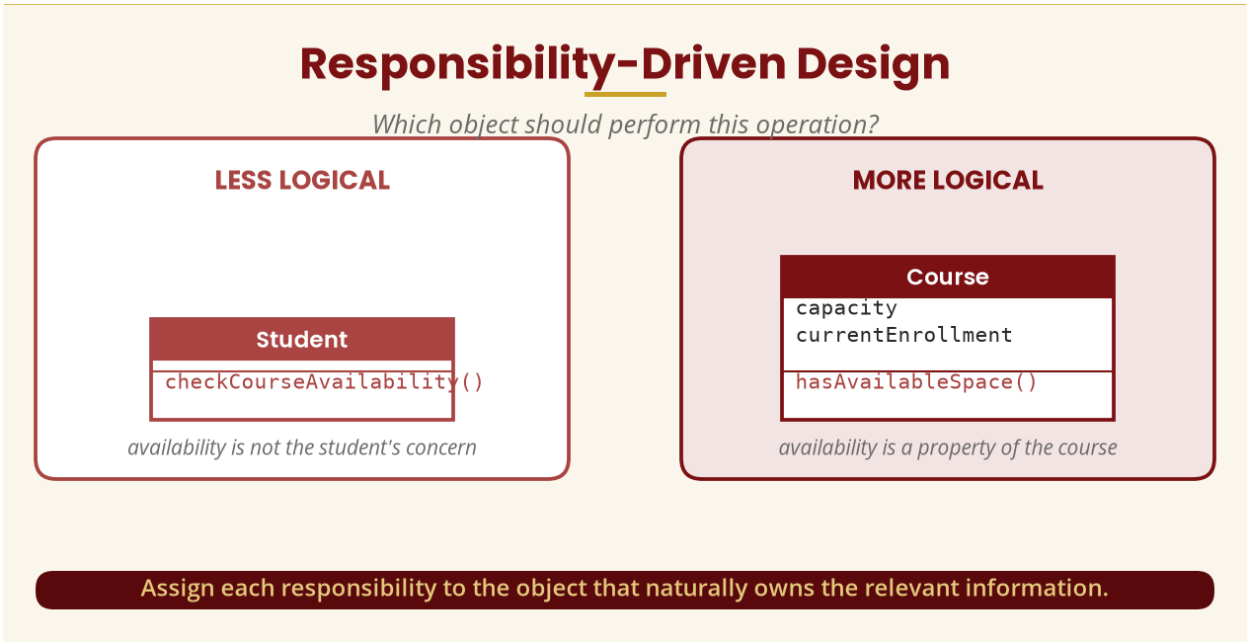

Figure 7 — Responsibility assignment

PART F & G — COHESION AND COUPLING

10. Cohesion

Cohesion asks how closely related the responsibilities inside a class or component are. **High cohesion**: a `Course` with courseCode, title, capacity, credits and hasSpace(), addStudent(), removeStudent() — everything about the course. **Low cohesion**: a `Course` that also registers students, sends email, calculates salary, generates PDFs, compresses images and backs up the database.

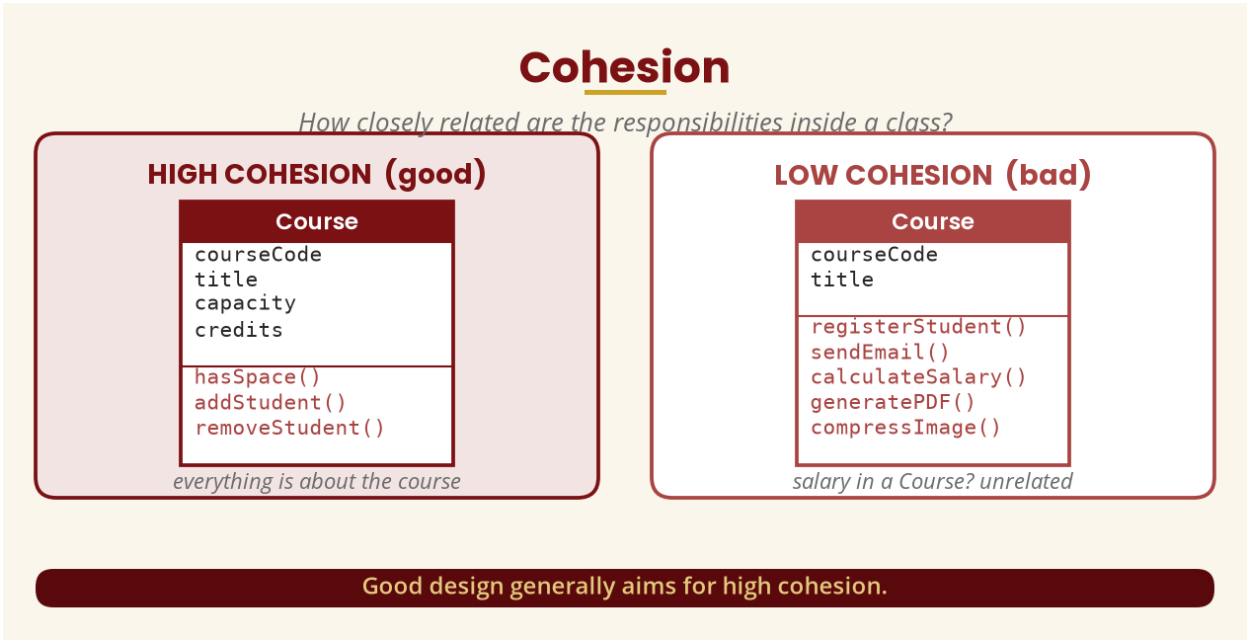

Figure 8 — High vs low cohesion

11. Coupling

Coupling describes the degree of dependency between components. High coupling — Student -> Payment -> Database -> Email -> SMS -> Reports — means changing the database affects everything. Lower coupling uses an abstraction: `PaymentService` depends on a `PaymentGateway` interface implemented by Bank and MobileMoney, so the service need not know every implementation detail.

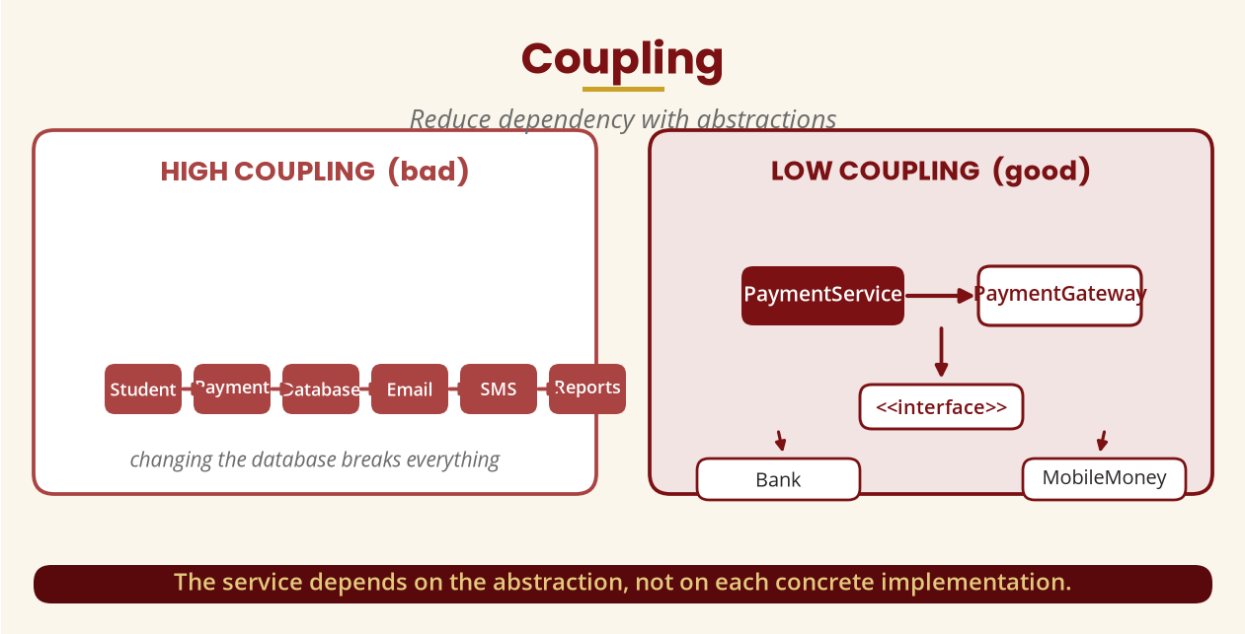

Figure 9 — High vs low coupling

PART H — INTERFACES

12. What Is an Interface?

An **interface** is essentially a contract: *"Any class implementing me must provide these operations."* For example `<<interface>>` `PaymentGateway` with `processPayment()` and `refundPayment()`, implemented by both `BankGateway` and `MobileGateway`. **Real-world analogy:** a phone charger doesn't care whether electricity came from hydro, solar, thermal or wind — it cares about the interface (the socket). The application similarly need not know which concrete payment provider implements the gateway.

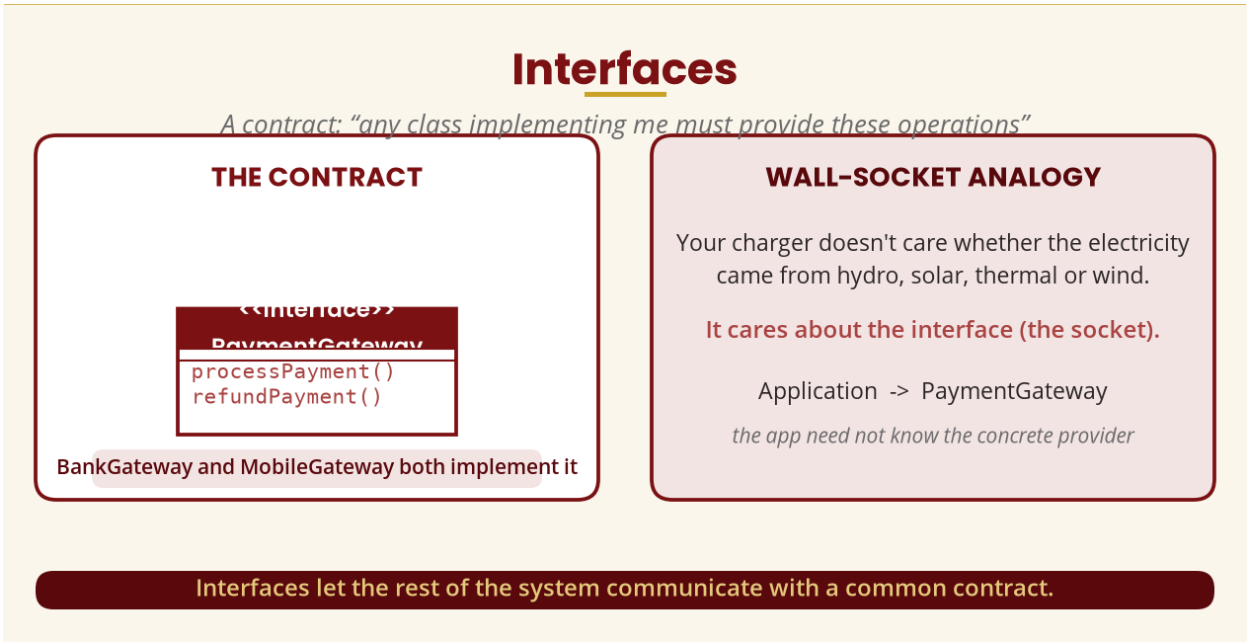

Figure 10 — Interfaces: contract and wall-socket analogy

PART I — ABSTRACT CLASSES

13. Abstract Class vs Interface

An **abstract class** is intended to provide common structure/behaviour for subclasses but is generally not instantiated directly — e.g. `Payment` (amount, date, process()) generalises `BankPayment` and `MobilePayment`. The comparison below is a classic examination area. Exact implementation possibilities depend on the language, so students should not assume Java, C# and C++ treat these constructs identically.

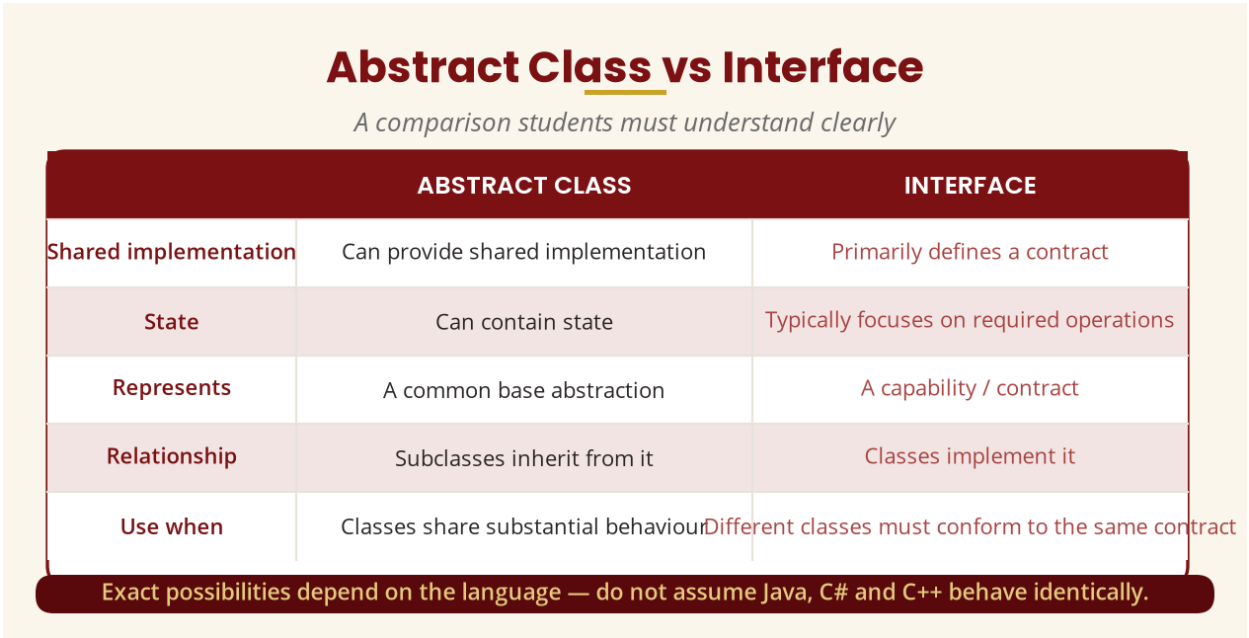

Figure 11 — Abstract class vs interface

PART J & K — ASSOCIATION VS INHERITANCE

14. Association vs Inheritance — the Easy Test

Association means objects/classes are related: *Student HAS/TAKES Course*. **Inheritance** means one class is a specialised form of another: *Student IS-A Person*. The easy test — **HAS / USES / KNOWS / INTERACTS WITH** signals association; **IS-A** signals inheritance. Never write `Student extends Course` just because both are classes: a Student is not a kind of Course.

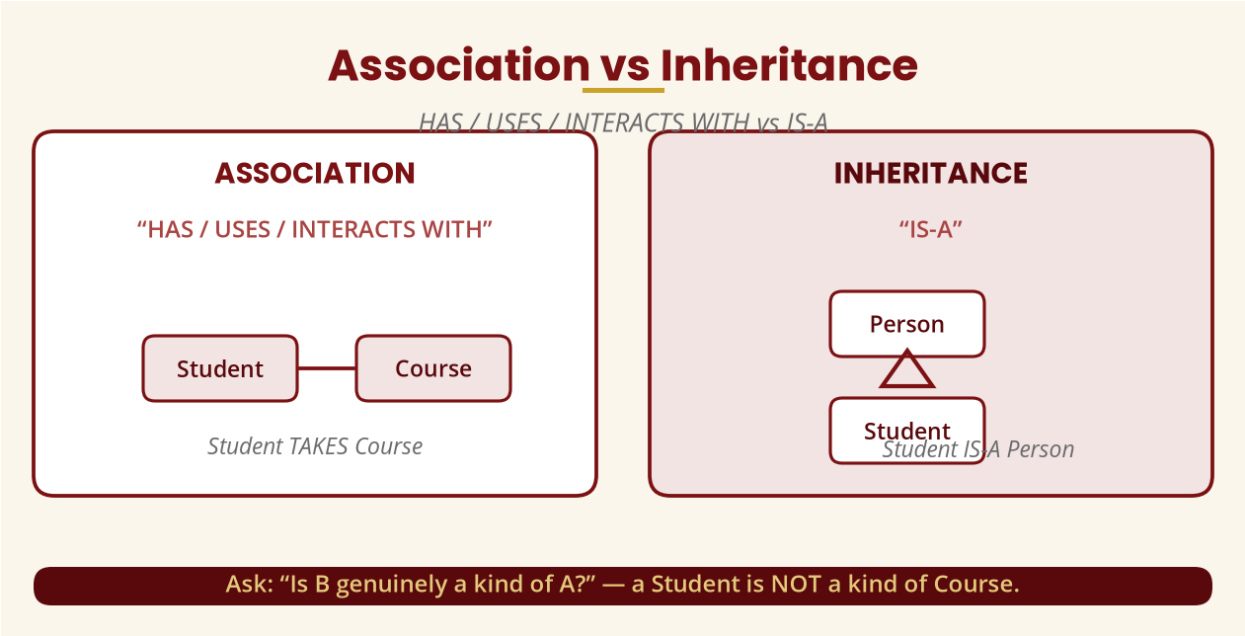

Figure 12 — Association vs inheritance

PART L & M — ENCAPSULATION, INFORMATION HIDING AND DEPENDENCIES

15. Encapsulation and Information Hiding

Encapsulation keeps data and the operations that manage it together while controlling access to internal state. In UML, `-balance : Decimal` means private; `account.withdraw(500)` lets the object enforce rules (`if balance < 500: reject`), whereas `account.balance = -500000` would let external code corrupt the state. **Information hiding** means hiding unnecessary implementation details.

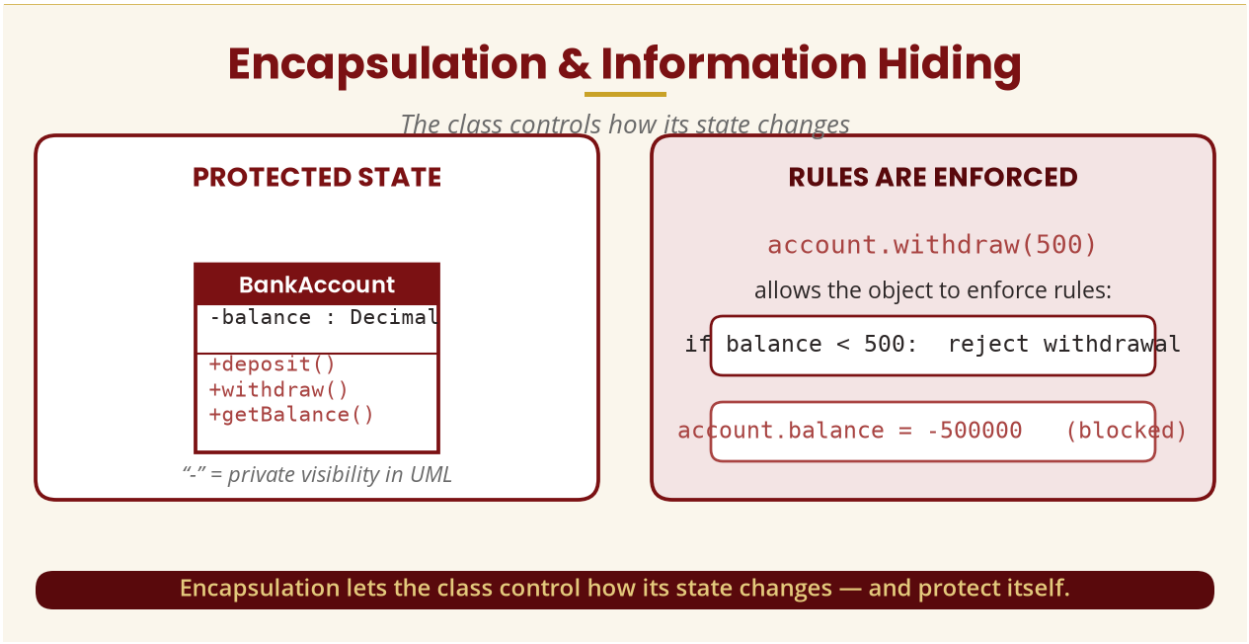

Figure 13 — Encapsulation and information hiding

16. Dependencies and Dependency Injection

A **dependency** exists when one component relies on another (RegistrationService depends on CourseRepository). **Dependency injection** provides the dependency instead of letting the service construct it — `RegistrationService(repository)` — improving testability, flexibility, substitution and maintainability.

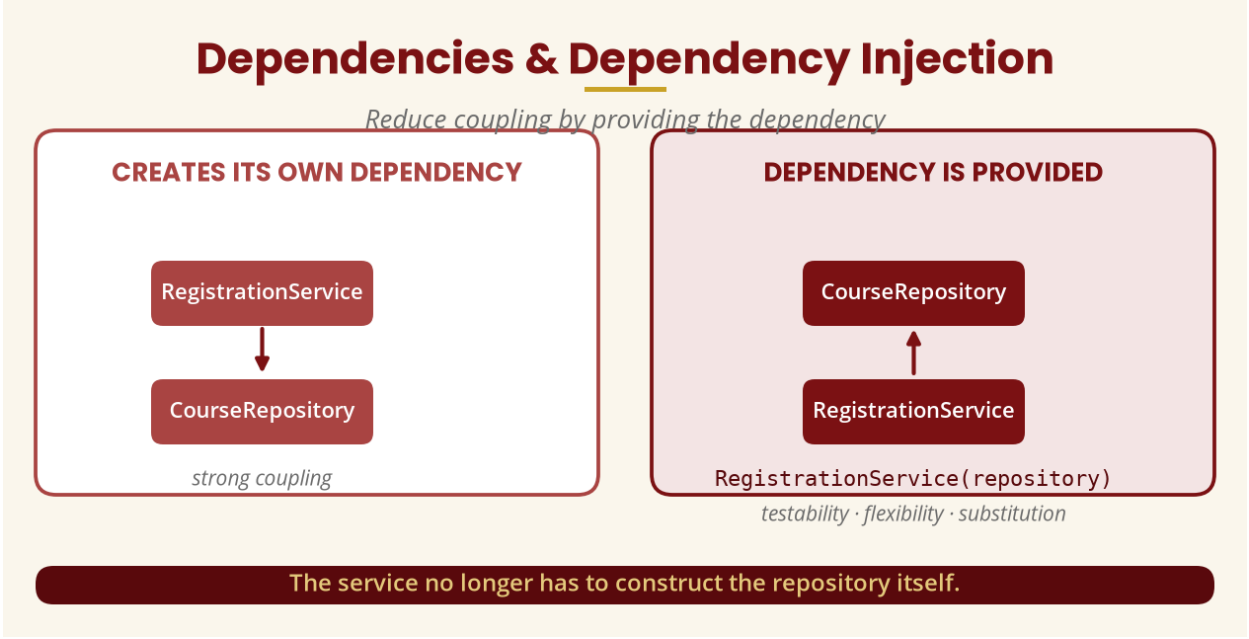

Figure 14 — Dependencies and dependency injection

PART N–Q — LAYERS, REUSE, CHANGE AND QUALITY

17. Separation of Concerns and Layered Design

A **concern** is a responsibility or area of functionality (authentication, payments, registration, reporting, notifications). Instead of one `StudentController` doing login(), registerCourse(), payFees(), sendSMS(), generateReport() and calculateResults(), we provide separate services. **Layered design** divides the system into logical levels — Presentation, Application/Service, Domain, Data Access, Database — so that if the web application becomes a mobile application, the business rules remain usable.

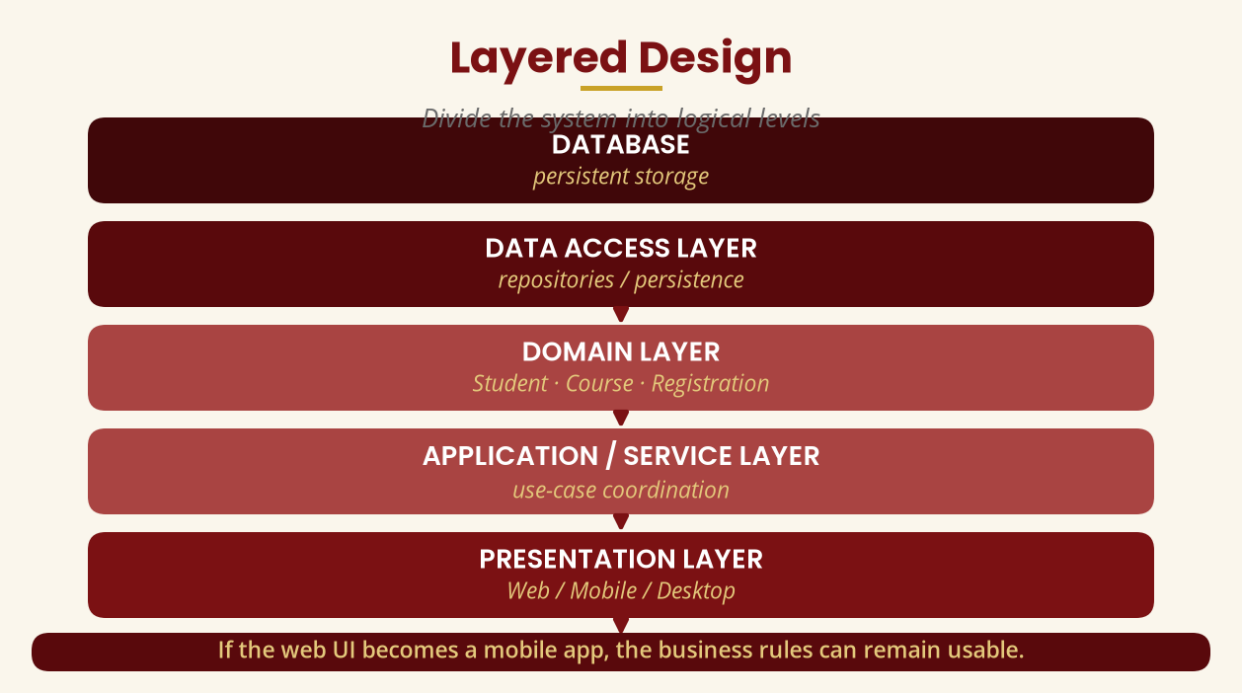

Figure 15 — Layered design

18. Design for Reuse and Design for Change

Reuse means designing software elements so they can be used again — a reusable `AuthenticationService` can serve the Student Portal, Staff Portal, Mobile App and Library System instead of four separate implementations. **Design for change** recognises that requirements change: a `NotificationService` behind a Notification interface lets you add Email, then SMS, then WhatsApp without rewriting the Student class.

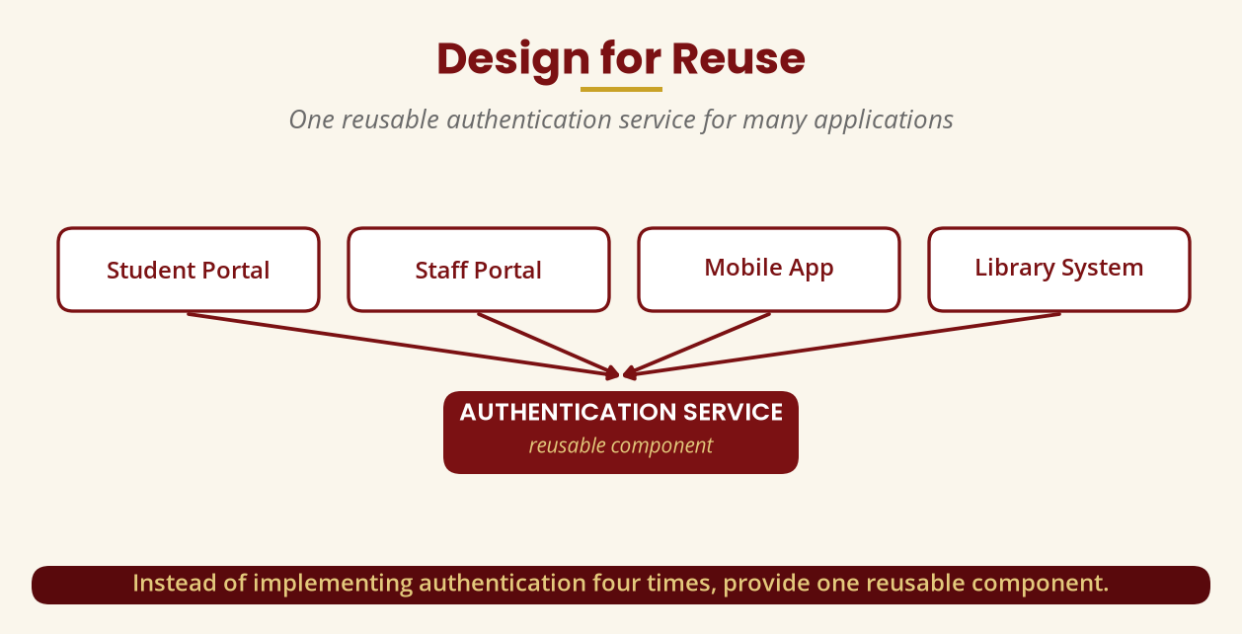

Figure 16 — Design for reuse

19. Judging a Design

Ask: is it **correct** (meets the requirements)? **understandable**? **maintainable**? **testable**? **reusable**? **flexible**? Is it **appropriately coupled** (no unnecessary dependencies)? Is it **cohesive** (each component has a focused purpose)?

PART T — DESIGN TRACEABILITY

20. Traceability

Traceability means being able to follow a requirement through the models into implementation and testing. Requirement **R01** ("students must be able to register for courses") traces through the Register Course use case, the registration activity and sequence diagrams, Student/Course/Registration, RegistrationService, `registerCourse()` and the automated test — so when someone asks *"Where is R01 implemented?"* we can trace it.

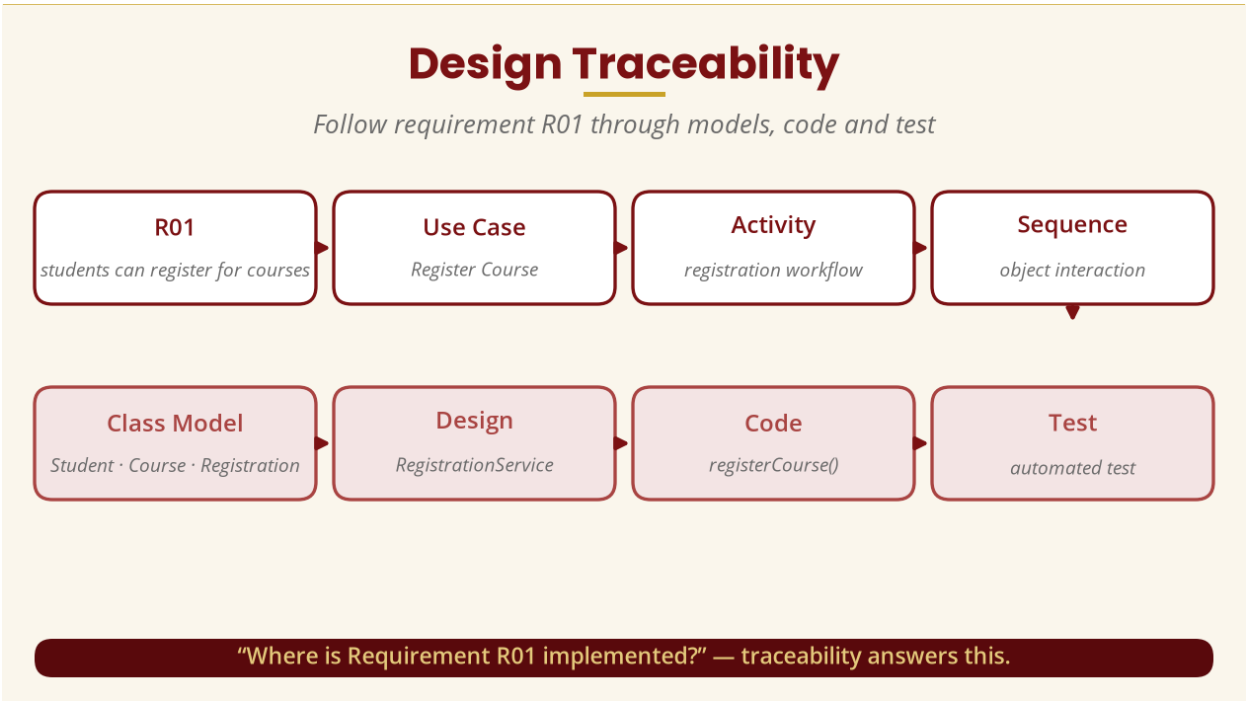

Figure 17 — Design traceability

PART U — INTEGRATED CASE STUDY

21. Online Course Registration — Seven Steps

(1) Use case — Student registers for a course. **(2) Identify classes** — domain classes Student, Course, Registration; design classes RegistrationService, AuthenticationService, CourseRepository, RegistrationRepository. **(3) Class design** — typed attributes and operations for Student, Course and Registration. **(4) Service** — RegistrationService coordinates across Student, Course and Registration. **(5) Sequence** — Student -> RegistrationService `register()` -> Course `hasSpace()` / `checkPrerequisite()` -> Registration created -> confirmation. **(6) State model** — Requested -> Validated -> Confirmed (or Rejected). **(7) Architecture** — Interface -> Services -> Domain -> Repositories -> Database.

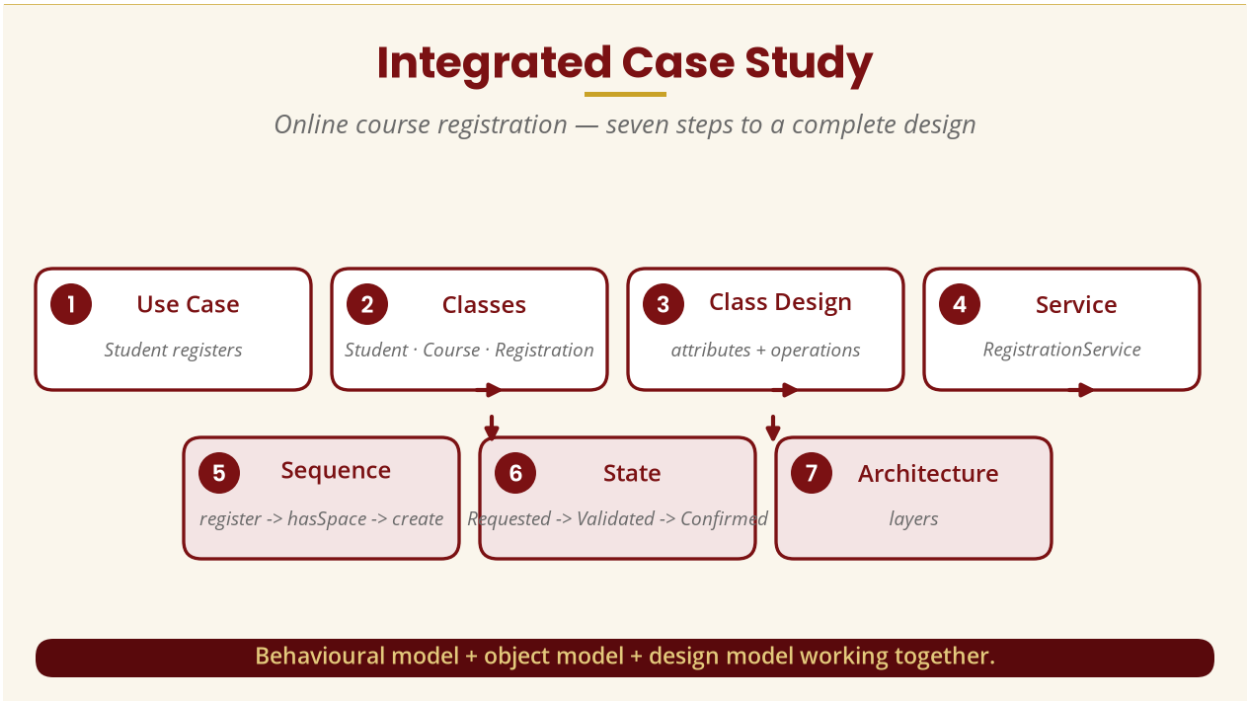

Figure 18 — The integrated case study

22. Common Design Mistakes

- **One giant class** (login, payments, registration, results, SMS, reports, database) — low cohesion and high coupling.
- **Everything inherits from everything** — only inherit when a meaningful IS-A relationship exists.
- **Direct database access everywhere** — route through repositories / data access.
- **Exposing everything** (public balance, public password) — undermines encapsulation.
- **Designing only for today** — consider reasonable future change without trying to predict everything.

23. Practical Class and Tutorial

Practical (1 hour): model an Online University Course Registration System and produce (A) a class diagram (Student, Course, Registration, RegistrationService), (B) a sequence diagram for registering a course, (C) a state diagram for the Registration lifecycle (Requested, Validated, Confirmed, Rejected, Cancelled), (D) an architecture diagram, and (E) an explanation of why each class and responsibility exists, where cohesion is high, where coupling is reduced, and where encapsulation and abstraction are used.

- **Tutorial Q1** — Explain the difference between an analysis class and a design class.
- **Tutorial Q2** — Critique a StudentSystem class (login, register, payFees, sendEmail, sendSMS, generateResults, connectDatabase): cohesion, why, five services, how the redesign improves it.
- **Tutorial Q3** — Explain association vs inheritance using Student, Course and Person.
- **Tutorial Q4** — Why are interfaces useful in OO design? Give a payment-system example.

24. Week 11 Summary and Key Terms

Concept	Simple explanation
Design refinement	Adding detail to an existing model.
Design class	A class detailed enough to support implementation.
Attribute / operation	Data/state of an object / something an object can do.
Responsibility	What an object knows or does.
Cohesion	How closely related responsibilities within a component are.
Coupling	How strongly components depend on one another.
Interface	A contract for behaviour.
Abstract class	A common base abstraction (not directly instantiated).
Encapsulation	Keeping internal state controlled.
Information hiding	Hiding unnecessary implementation details.
Dependency injection	Providing a dependency instead of constructing it.
Layering	Organising software into logical levels.
Reuse	Designing components that can be used again.
Traceability	Following requirements through models and implementation.

THE MOST IMPORTANT IDEA OF WEEK 11

Drawing a UML class diagram is **not** the same as completing an OO design. The goal is not simply to create more diagrams — it is to create a design that is **correct, understandable, maintainable, testable, reusable and adaptable**. Path: requirement -> analysis (Student, Course...) -> refinement (attributes + operations) -> responsibilities -> interfaces + dependencies -> cohesion + coupling decisions -> architecture -> implementable design.

25. Examination-Style Questions

Essay (25 marks). Explain the process of refining an analysis model into an object-oriented design model, illustrating with an Online Student Registration System. Discuss analysis classes, design classes, attributes, operations, responsibilities, relationships, interfaces, cohesion, coupling, encapsulation, architecture and traceability.

Essay (20 marks). "A good object-oriented design should have high cohesion and low coupling." Discuss this statement using suitable examples.

Essay (20 marks). Explain the difference between association, inheritance and interface implementation, using UML examples.

Scenario (25 marks). A university wants an online payment system supporting bank, mobile-money and card payments. Design an OO solution that allows new payment methods to be added with minimal changes — provide a class/interface model, relationships, responsibilities, and an explanation of coupling and how the design supports reuse and change.

26. References and Further Reading

Authoritative / current online sources

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1.* OMG.
- Noback, M. (2018). *Principles of Package Design: Creating Reusable Software Components.* Apress/Springer.
- Gonzalez, P. (2025). *Clean Apex Code: Software Design for Salesforce Developers.* Springer — OO design, modularity, coupling, cohesion, testing, dependency injection, maintainability.
- *Recognising Object-Oriented Software Design Quality: A Practitioner-Based Questionnaire Survey.* Software Quality Journal, Springer.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*